FORM, Monte Carlo e amostragem por importância: revisão do código, um talude infinito sob chuva e o uso no talude com campos aleatórios

14 de novembro de 2026

Sumário

1 Resumo **1**

2 Formulação **2**
2.1 Probabilidade de falha . . . 2
2.2 Transformação para o espaço normal padrão (Nataf) . . . 3
2.3 FORM . . . 3
2.4 Monte Carlo . . . 4
2.5 Amostragem por importância . . . 4

3 Revisão do código original **4**

4 O código revisado **5**

5 Problema 1: $g = X_1X_2 - X_3$ **6**

6 Problema 2 (proposto): talude infinito não saturado sob chuva **7**
6.1 Modelo . . . 7
6.2 Dois cenários . . . 9
6.3 O efeito dos erros do código original . . . 10
6.4 HL-RF com e sem a busca de Armijo . . . 10
6.5 Curva de fragilidade . . . 11

7 Uso no talude com chuva e campos aleatórios **11**
7.1 A correspondência . . . 11
7.2 Qual método e quando . . . 12
7.3 Amostragem por importância sem gradiente: o caminho proposto . . . 12

8 Conclusões **13**

A Como reproduzir **14**

B O código revisado **14**

1 Resumo

O código original (Mathematica) calcula a probabilidade de falha do estado limite $g = X_1X_2 - X_3$, com três normais independentes, de três formas: o FORM, pelo ponto de projeto obtido com `NMinimize`; Monte Carlo com $N = 1000$; e amostragem por importância, com a densidade centrada no ponto de projeto. O ponto

de projeto está certo ($\beta = 3{,}049$), mas a revisão encontrou erros que o exemplo não deixa aparecer, porque nele as variáveis são normais e não correlacionadas:

- a transformação para o espaço normal padrão independente troca L por L^{-1} (Seção 3). Com correlação, isso muda o resultado: no problema proposto, o β cai de 3,541 para 2,511, e o P_f do FORM fica 30,2 vezes maior;
- a volta $z \to x$ está errada: a saída `{160.+X1, 75.+X2, 199000.+X3}` do próprio caderno mostra isso;
- Monte Carlo com $N = 1000$ não estima $P_f \approx 1{,}2 \times 10^{-3}$: o coeficiente de variação é de 92 %, e 31 % das rodadas não têm nenhuma falha ($P_f = 0$ e $\beta = \infty$);
- o ponto de projeto da amostragem por importância foi digitado à mão, e falta o coeficiente de variação da estimativa.

O código revisado (`confiabilidade.wl`) tem:

- a transformação de Nataf com marginais quaisquer e correlação;
- o FORM pelo algoritmo HL-RF, com busca de Armijo, gradiente por diferenças e fatores de importância;
- Monte Carlo e amostragem por importância vetorizados, com o coeficiente de variação;
- as funções originais, corrigidas.

Os números deste documento vêm de uma versão em Python com o mesmo algoritmo (`python/`). Não há Mathematica nesta máquina. O código Mathematica rodou no Mathics, um interpretador livre da linguagem, com substitutos para as funções estatísticas, e deu o mesmo β (Seção 4).

O problema proposto (Seção 6) é um talude infinito não saturado sob chuva, com os parâmetros do talude de Vargas Ceron et al. (2025): c' e ϕ' lognormais, correlacionados, e a carga de pressão no plano de ruptura como a variável da chuva. Ele usa as partes do código que o exemplo original não usa (marginais lognormais e correlação). Os resultados principais:

- a curva de fragilidade $P_f(\mu_h)$ vai de 10^{-12} a 0,6;
- o FORM superestima P_f em 6 a 42 %, mais nos cenários secos;
- o HL-RF sem a busca de Armijo não converge quando $P_f < 10^{-5}$;
- a amostragem por importância com 1000 amostras acompanha a referência em toda a curva.

A Seção 7 mostra como usar isso no talude com campos aleatórios (`SlopeMonteCarlo`): os ξ_k da expansão de Karhunen-Loève já são o espaço normal padrão, e $G = \Gamma - 1$. Para o $P_f \approx 11$ % do artigo, Monte Carlo basta. A amostragem por importância passa a compensar abaixo de $P_f \approx 10^{-2}$, com o deslocamento estimado das próprias realizações que falham. O FORM direto custaria $2n$ análises por iteração, com $n = 6000$.

2 Formulação

2.1 Probabilidade de falha

Com as variáveis aleatórias $\boldsymbol{X}$, de densidade conjunta $f_{\boldsymbol{X}}$, e a função de estado limite g (falha quando $g \le 0$),

$$P_f = P[g(\boldsymbol{X}) \le 0] = \int_{g(\boldsymbol{x}) \le 0} f_{\boldsymbol{X}}(\boldsymbol{x})\, d\boldsymbol{x}, \qquad \beta = -\Phi^{-1}(P_f), \tag{1}$$

em que Φ é a distribuição normal padrão. O índice β é o equivalente normal de P_f.

2.2 Transformação para o espaço normal padrão (Nataf)

Três espaços (as convenções do código):

- $\boldsymbol{X}$, as variáveis físicas, com marginais F_i e correlação R_X;
- $\boldsymbol{Z}$, normais padrão correlacionadas, $z_i = \Phi^{-1}(F_i(x_i))$, com correlação R_Z;
- $\boldsymbol{U}$, normais padrão independentes: $\boldsymbol{z} = L\boldsymbol{u}$ e $\boldsymbol{u} = L^{-1}\boldsymbol{z}$, com $R_Z = LL^T$ (Cholesky, L triangular inferior).

No espaço $\boldsymbol{U}$, a função de estado limite é $G(\boldsymbol{u}) = g(\boldsymbol{x}(\boldsymbol{u}))$.

O modelo de Nataf escolhe R_Z de modo que $\boldsymbol{X}$ tenha a correlação pedida R_X (Liu e Der Kiureghian, 1986):

$$\rho_{X,ij} = \iint \frac{x_i(z_i) - \mu_i}{\sigma_i} \frac{x_j(z_j) - \mu_j}{\sigma_j} \varphi_2(z_i, z_j; \rho_{Z,ij})\, dz_i\, dz_j. \tag{2}$$

Para as marginais deste documento, há formas fechadas, com $\delta = \sigma/\mu$ e $\zeta = \sqrt{\ln(1+\delta^2)}$:

- normal–normal: $\rho_Z = \rho_X$;
- lognormal–lognormal: $\rho_Z = \ln(1 + \rho_X \delta_i \delta_j)/(\zeta_i \zeta_j)$;
- normal–lognormal: $\rho_Z = \rho_X \delta_j / \zeta_j$.

Para outras marginais, o código resolve a integral numericamente. A transformação de cada marginal é a exata: $x = \exp(\lambda + \zeta z)$ na lognormal, com $\lambda = \ln\mu - \zeta^2/2$.

O código original usa a normal equivalente de Rackwitz e Fiessler (1978): no ponto x^*, a normal com a mesma densidade e a mesma distribuição acumulada. Na lognormal, ela é

$$\sigma^N = x^*\zeta, \qquad \mu^N = x^*(1 - \ln x^* + \lambda).$$

Daí vêm duas observações:

- $(x^* - \mu^N)/\sigma^N = (\ln x^* - \lambda)/\zeta$: no próprio ponto, a normal equivalente coincide com a transformação exata, e as duas levam ao mesmo ponto de projeto se a normal equivalente for refeita a cada iteração;
- com correlação, a correlação a usar entre as normais é R_Z, e não R_X. O código original usa R_X.

2.3 FORM

O FORM aproxima a superfície $G(\boldsymbol{u}) = 0$ pelo plano tangente no ponto de projeto. O ponto de projeto é o ponto da superfície mais próximo da origem (Hasofer e Lind, 1974):

$$\boldsymbol{u}^* = \arg\min\{\|\boldsymbol{u}\| : G(\boldsymbol{u}) = 0\}, \quad \beta = \boldsymbol{\alpha}\cdot\boldsymbol{u}^*, \quad \boldsymbol{\alpha} = -\frac{\nabla G(\boldsymbol{u}^*)}{\|\nabla G(\boldsymbol{u}^*)\|}, \quad P_f \approx \Phi(-\beta). \tag{3}$$

Os fatores de importância α_i^2 somam 1. Cada um é a fração da variância do G linearizado que vem de u_i.

O HL-RF itera

$$\boldsymbol{u}_{k+1}^{\mathrm{HL}} = \frac{\nabla G_k \cdot \boldsymbol{u}_k - G_k}{\|\nabla G_k\|^2} \nabla G_k. \tag{4}$$

Esse é o ponto de projeto do plano tangente em $\boldsymbol{u}_k$. O HL-RF puro dá o passo inteiro e pode oscilar sem convergir (Tabela 5). O código revisado dá o passo $\boldsymbol{u}_{k+1} = \boldsymbol{u}_k + \lambda \boldsymbol{d}_k$, com $\boldsymbol{d}_k = \boldsymbol{u}_{k+1}^{\mathrm{HL}} - \boldsymbol{u}_k$ e $\lambda = 1, \frac{1}{2}, \frac{1}{4}, \ldots$. Ele escolhe o maior λ que reduz a função de mérito (Zhang e Der Kiureghian, 1995)

$$m(\boldsymbol{u}) = \tfrac{1}{2}\|\boldsymbol{u}\|^2 + c\,|G(\boldsymbol{u})|, \qquad c = \frac{2\|\boldsymbol{u}_k\| + 1}{\|\nabla G_k\|} > \frac{\|\boldsymbol{u}_k\|}{\|\nabla G_k\|}, \tag{5}$$

pela condição de Armijo: $m(\boldsymbol{u}_k + \lambda \boldsymbol{d}_k) \le m(\boldsymbol{u}_k) + 10^{-4}\lambda\, \nabla m \cdot \boldsymbol{d}_k$. Com esse c, $\boldsymbol{d}_k$ é direção de descida de m.

Critérios de parada: $|G|/|G(\boldsymbol{0})| < 10^{-6}$ e $\|\boldsymbol{u} - (\boldsymbol{\alpha}\cdot\boldsymbol{u})\boldsymbol{\alpha}\| < 10^{-4}$. O segundo diz que $\boldsymbol{u}$ é paralelo ao gradiente. O gradiente é calculado por diferenças centrais e custa $2n$ avaliações de G por iteração. Num modelo de elementos finitos, cada avaliação é uma análise.

2.4 Monte Carlo

Com N amostras $\boldsymbol{u}_k \sim \mathcal{N}(\mathbf{0}, I)$,

$$\hat{P}_f = \frac{1}{N}\sum_{k=1}^{N} I[G(\boldsymbol{u}_k) \le 0], \qquad \mathrm{CoV}(\hat{P}_f) = \sqrt{\frac{1-P_f}{NP_f}}, \qquad N \approx \frac{1-P_f}{P_f\,\delta^2} \text{ para um CoV } \delta. \tag{6}$$

A probabilidade de nenhuma das N amostras falhar é $(1-P_f)^N$.

2.5 Amostragem por importância

As amostras vêm de uma densidade h concentrada perto da falha e são pesadas pela razão das densidades:

$$\begin{aligned} P_f &= \mathrm{E}_h\left[I[G(\boldsymbol{v}) \le 0]\,\frac{\varphi_n(\boldsymbol{v})}{h(\boldsymbol{v})} \right], \qquad \hat{P}_f = \frac{1}{N}\sum_{k=1}^{N} q_k, \\ q_k &= I[G(\boldsymbol{v}_k) \le 0]\, w(\boldsymbol{v}_k), \qquad \mathrm{CoV}(\hat{P}_f) = \frac{s_q}{\sqrt{N}\,\hat{P}_f}, \end{aligned} \tag{7}$$

em que s_q é o desvio padrão amostral dos q_k. Com h a normal padrão deslocada para o ponto de projeto, $h(\boldsymbol{v}) = \varphi_n(\boldsymbol{v} - \boldsymbol{u}^*)$, o peso é

$$w(\boldsymbol{v}) = \exp\!\left(-\boldsymbol{v}\cdot\boldsymbol{u}^* + \tfrac{1}{2}\|\boldsymbol{u}^*\|^2\right). \tag{8}$$

O peso só depende de $\boldsymbol{v}$ pela projeção $\boldsymbol{v}\cdot\boldsymbol{u}^*$. Com $\boldsymbol{v} \sim h$, $\mathrm{E}_h[w^2] = e^{\beta^2}$, qualquer que seja a dimensão n. Por isso essa amostragem funciona também com milhares de variáveis, como os ξ_k de uma expansão de Karhunen-Loève (Au e Beck, 2003). As condições são duas:

- o deslocamento tem de estar certo: um deslocamento $\boldsymbol{u}^* + \boldsymbol{\delta}$, com ruído $\boldsymbol{\delta}$ em direções que não importam, leva a $\mathrm{E}_h[w^2] = e^{\beta^2 + \|\boldsymbol{\delta}\|^2}$;
- a covariância de h não pode mudar.

A amostragem do código original, normal em $\boldsymbol{X}$ centrada em $\boldsymbol{x}^*$ e com os mesmos desvios, é esta mesma quando as variáveis são normais independentes.

3 Revisão do código original

A Tabela 1 lista o que foi corrigido. Os itens 1 a 3 são erros que mudam o resultado com correlação. Os demais tornam o código frágil, impreciso ou não reprodutível.

Tabela 1: Erros e fragilidades do código original e as correções.

	Trecho	Problema	Correção
1	`FromXtoY`: `Jyx = Jzy.Jzx, Jxy = Jxz.Jyz`	L e L^{-1} trocados: $\boldsymbol{y} = L\boldsymbol{z}$ em vez de $L^{-1}\boldsymbol{z}$. Com $R_X = I$ não aparece. Com $\rho = -0{,}5$, as Z implícitas ficam com correlação $+0{,}459$ e desvio 1,314 (Seção 6.3).	`Jyx = Jyz.Jzx, Jxy = Jxz.Jzy`
2	`Jxz.(z + Mneq)`	a volta $z \to x$ está errada: o caderno mostra `{160.+X1, ...}`.	$\boldsymbol{x} = J_{xz}\boldsymbol{z} + \boldsymbol{\mu}^N$ (`FromZtoX`)
3	`FromYtoZ: {Jzy, Jyz} = ComputeJYZ[Rx]`	os nomes saem trocados. O resultado ($L\boldsymbol{y}$) está certo por acaso e é incoerente com o item 1: a ida e volta $x \to y \to z$ dá $L^2\boldsymbol{z}$.	nomes na ordem de `ComputeJYZ`
4	`Det[L.Transpose[L] - RX] > 10^-8`	o determinante não mede o erro: uma matriz com um único termo errado tem determinante nulo.	`Max[Abs[...]]`
5	`If[NRV == 1, {1, 1}], If[Det[RX] != 1, ...]`	escalares no lugar de matrizes 1×1 (o produto `Dot` falha). O teste da identidade compara um determinante em ponto flutuante com 1.	Cholesky sempre (a da identidade é a identidade)
6	correlação usada no espaço normal	é R_X, a das variáveis físicas; com marginais lognormais, o certo é R_Z. Aqui o erro é pequeno ($-0{,}5 \to -0{,}5161$), mas cresce com o CoV.	correlação de Nataf (`rhoZ`)
7	`g[y_] := (5 y1 + 40)...` e depois `g[X_] := X[[1]] X[[2]] - X[[3]]`	a primeira não usa o argumento: funciona porque é chamada com as variáveis globais. A segunda redefine o mesmo símbolo. O FORM não usa as funções de transformação (g foi escrita à mão em y).	`gX[{x1_, x2_, x3_}]` e `G[u] = gX[xDeU[mod, u]]`
8	Monte Carlo com $N = 1000$	com $P_f \approx 1{,}2 \times 10^{-3}$, o CoV é de 92 %, e 31 % das rodadas dão $\hat{P}_f = 0$ e `InverseCDF[..., 0]` $= -\infty$. Para um CoV de 10 %, são necessárias $8{,}5 \times 10^4$ amostras.	CoV impresso, β só se $0 < \hat{P}_f < 1$
9	amostragem por importância	o ponto de projeto foi digitado à mão (`-2.28993 sig1 + mu1`), assim como a linha do FORM no gráfico (`-3.04`). Falta o CoV. A linha comentada `w = fX/@data/hX/@xdatafailzone` mistura amostras de densidades diferentes, e `w/Norm[w]` não é a normalização da amostragem autonormalizada ($\sum w$).	`importancia[G, f["u"], N]` com o CoV
10	sem `SeedRandom`	resultados não reprodutíveis.	`SeedRandom[20261114]`
11	`NMinimize` para o ponto de projeto	funciona aqui, mas não dá $\boldsymbol{\alpha}$. Com g de um modelo externo, faria muitas avaliações sem usar o gradiente.	HL-RF com Armijo

4 O código revisado

O arquivo `confiabilidade.wl` abre no Mathematica como um caderno de código (*Run All Code*). As funções:

- `lnDist[média, CoV]`: a lognormal pela média e pelo CoV;
- `xDeZ`, `zDeX`: as transformações exatas de uma marginal. A normal e a lognormal são vetorizadas; as outras, ponto a ponto, com a cauda superior pela função de sobrevivência;
- `rhoZ`: a correlação de Nataf;
- `modelo[dists, RX]`: guarda R_Z e L;
- `xDeU`, `xDeUs`, `uDeX`: as transformações de um ponto, de muitas amostras e a inversa;
- `form[G, n]`: o HL-RF com Armijo. As opções são `"Passo"`, `"TolG"`, `"TolU"`, `"MaxIter"` e `"Armijo"`. Devolve β, $\boldsymbol{u}^*$, $\boldsymbol{\alpha}$, P_f, as iterações, as avaliações de G e o histórico;
- `monteCarlo[Gv, n, N]`, `importancia[Gv, u*, N]`: as estimativas com o CoV. `Gv` recebe uma matriz de amostras;
- a Seção 6 do arquivo tem as funções originais corrigidas e um teste de ida e volta com correlação;

- a Seção 7 tem o esboço de um G externo.

A função de estado limite é escrita uma vez, nas variáveis físicas, com o padrão `{x1_, x2_, x3_}`. Ela vale para um ponto e também para as colunas de muitas amostras (`gX[Transpose[xs]]`).

Verificação. O código Mathematica não rodou no Mathematica: não há licença nesta máquina. Ele rodou no Mathics 10, um interpretador livre da linguagem. O Mathics não tem as funções estatísticas, então o teste usa substitutos (`mathics/shims.m`): `CDF`, `PDF` e `InverseCDF` da normal, a média e o desvio da lognormal, `RandomVariate` e `CholeskyDecomposition`. O script `mathics/roda_mathics.sh` repete o teste em cerca de 1 min. Os resultados:

- **Problema 1.** O FORM deu $\beta = 3{,}04907$ em 6 iterações e 43 avaliações, com $\boldsymbol{\alpha}^2 = (0{,}564; 0{,}049; 0{,}387)$, os mesmos números da versão em Python. A amostragem por importância com 1000 amostras deu $1{,}20 \times 10^{-3}$, com CoV de 6 %.
- **Problema 2.**
  - $\rho_Z = -0{,}516123$.
  - O FORM com Armijo deu $\beta = 5{,}19746$, 3,54141 e 1,07813 para $\mu_h = -3$, -2 e $-0{,}5$ m, nas mesmas 12, 21 e 7 iterações do Python.
  - O HL-RF puro não convergiu em $\mu_h = -3$ m: depois de 100 iterações, $\beta = 4{,}586$, como no Python.
  - A G vetorizada coincide com a pontual.
  - A amostragem por importância no cenário A, com 400 amostras, deu $1{,}55 \times 10^{-4}$ (CoV de 10 %).
- **Funções originais corrigidas.** A ida e volta $x \to y \to z \to x$ com correlação devolve o x de partida.
- **Erro de sintaxe.** O teste achou um, já corrigido: o `u*)` dentro de um comentário fechava o comentário antes da hora.

Ficaram de fora do teste no Mathics as integrais de referência (`NIntegrate`), os gráficos e as tabelas dos cenários, que no Mathics levariam horas. As referências do documento são as da versão em Python. Os números das próximas seções são da versão em Python (`python/confiab.py` e `python/problemas.py`), que tem o mesmo algoritmo. As referências são integrais calculadas com quadratura adaptativa.

5 Problema 1: $g = X_1 X_2 - X_3$

$X_1 \sim \mathcal{N}(40; 5)$, $X_2 \sim \mathcal{N}(50; 2{,}5)$ e $X_3 \sim \mathcal{N}(1000; 200)$, independentes. É o exemplo clássico da resistência $F_y Z$ contra o momento M. A referência é $P_f = \mathrm{E}[\Phi((1000 - X_1 X_2)/200)]$, integrada em X_1 e X_2.

Tabela 2: Problema 1: referência, FORM, Monte Carlo e amostragem por importância.

Método	N (avaliações de g)	P_f	β	CoV estimado	erro de P_f
Referência (integração numérica)	—	$1{,}177 \times 10^{-3}$	3,0415	—	—
FORM, HL-RF (diferenças centrais)	43	$1{,}148 \times 10^{-3}$	3,0491	—	−2,5 %
Monte Carlo	10^3	$1{,}00 \times 10^{-3}$	3,090	99,9 %	−15,0 %
Monte Carlo	10^4	$1{,}30 \times 10^{-3}$	3,011	27,7 %	10,5 %
Monte Carlo	10^5	$1{,}14 \times 10^{-3}$	3,051	9,4 %	−3,1 %
Monte Carlo	10^6	$1{,}18 \times 10^{-3}$	3,041	2,9 %	0,2 %
Amostragem por importância	10^3	$1{,}212 \times 10^{-3}$	3,0327	6,1 %	3,0 %
Amostragem por importância	10^4	$1{,}151 \times 10^{-3}$	3,0483	1,9 %	−2,2 %

- **FORM.** $\beta = 3{,}04907$, o mesmo do `NMinimize` original, em 6 iterações (43 avaliações de g).
  - Ponto de projeto: $\boldsymbol{u}^* = (-2{,}2899; -0{,}6767; 1{,}8961)$ e $\boldsymbol{x}^* = (28{,}55; 48{,}31; 1379{,}23)$.

  - Fatores de importância: $\boldsymbol{\alpha}^2 = (0{,}564; 0{,}049; 0{,}387)$. A resistência X_1 e o momento X_3 dominam, e X_2 quase não pesa.
  - O FORM erra $-2{,}5\,\%$ em P_f: a superfície $x_1 x_2 = x_3$ é curva.

- **Monte Carlo com $N = 1000$.** Em 2000 repetições, $31\,\%$ deram $\hat{P}_f = 0$ (o esperado é $(1 - P_f)^{1000} = 31\,\%$). Os percentis de 5 e 95 % são 0 e $3{,}0 \times 10^{-3}$.
- **Amostragem por importância com $N = 1000$.** O CoV é de $5{,}9\,\%$, e os percentis são $1{,}06 \times 10^{-3}$ e $1{,}29 \times 10^{-3}$. Para o mesmo CoV, Monte Carlo precisaria de cerca de 250 vezes mais amostras (Figura 1b).

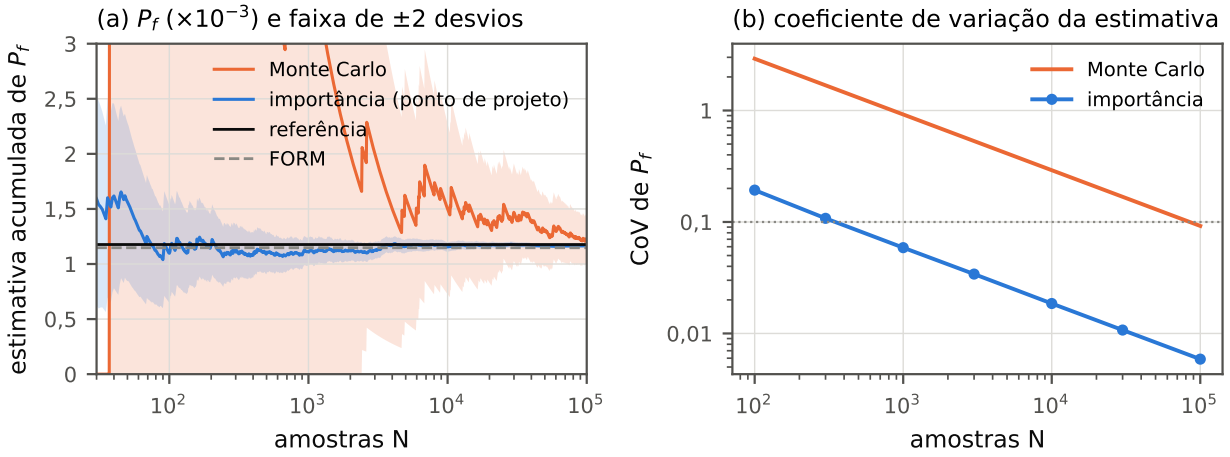


Figura 1: Problema 1. (a) A estimativa acumulada de P_f, com a faixa de ± 2 desvios: Monte Carlo e amostragem por importância. (b) O CoV da estimativa contra o número de amostras: a fórmula para Monte Carlo e a média de 20 rodadas para a amostragem por importância.

6 Problema 2 (proposto): talude infinito não saturado sob chuva

6.1 Modelo

O plano de ruptura é paralelo à superfície, à profundidade vertical z (Figura 2). Com a inclinação β_s e o peso específico γ, as tensões no plano são

$$\sigma_n = \gamma z \cos^2 \beta_s, \qquad \tau = \gamma z \sin \beta_s \cos \beta_s.$$

A chuva entra pela carga de pressão h no plano: a pressão da água é $u_w = \gamma_w h$, com $h < 0$ sucção e $h > 0$ pressão positiva (lençol suspenso). Com a tensão efetiva de Bishop, $\sigma' = \sigma_n - \chi u_w$ e $u_a = 0$, o fator de segurança é

$$FS = \frac{c' + (\gamma z \cos^2 \beta_s - \chi\, \gamma_w h) \tan \phi'}{\gamma z \sin \beta_s \cos \beta_s}, \qquad g = FS - 1. \tag{9}$$

Usei $\chi = 1$ ($\phi^b = \phi'$), como no material do `SlopeRandomField(p.chi = 1)`. O código também aceita $\chi = S_e(h)$ de van Genuchten, que é a tensão de sucção de Lu e Godt (2008), com a resistência por sucção que satura.

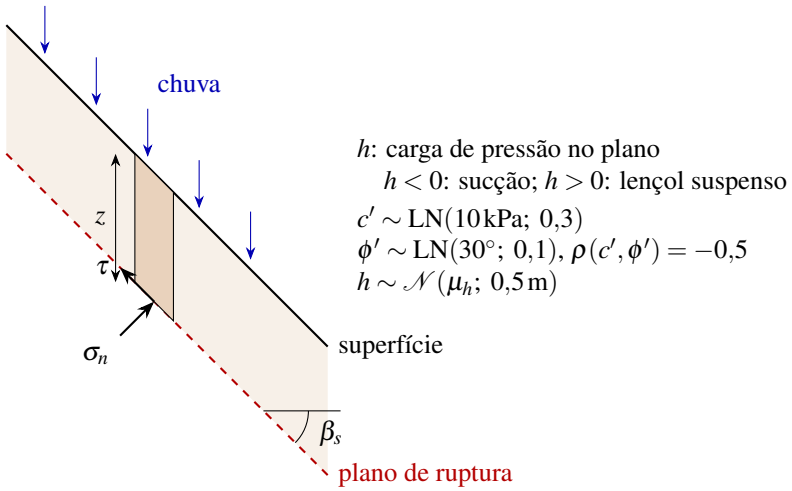


Figura 2: O talude infinito com o plano de ruptura à profundidade z e as variáveis aleatórias.

Parâmetros. Os do talude de Vargas Ceron et al. (2025), Tabela 2, e os do `SlopeMonteCarlo`:

- geometria e solo: $\beta_s = 45^\circ$, $\gamma = 20\,\mathrm{kN/m^3}$ e $\gamma_w = 9{,}81\,\mathrm{kN/m^3}$, com $z = 2\,\mathrm{m}$. Daí $\sigma_n = \tau = 20\,\mathrm{kPa}$;
- variáveis aleatórias:
  - $c' \sim \mathrm{LN}(10\,\mathrm{kPa};\ \mathrm{CoV}\ 0{,}3)$;
  - $\phi' \sim \mathrm{LN}(30^\circ;\ \mathrm{CoV}\ 0{,}1)$;
  - $h \sim \mathcal{N}(\mu_h;\ 0{,}5\,\mathrm{m})$;
- correlação: $\rho(c', \phi') = -0{,}5$ nas variáveis físicas, o que dá $\rho_Z = -0{,}5161$ pela Seção 2.2. O modelo de elementos finitos usa campos independentes, e os dois casos estão na Tabela 4.

A chuva. A média μ_h é o cenário: $\mu_h = -2\,\mathrm{m}$ (sucção de cerca de 20 kPa) antes da chuva, e $\mu_h \to 0$ ou positivo depois. O desvio de h resume, numa variável, a incerteza da resposta hidráulica, que no modelo de elementos finitos vem do campo de k_v (CoV de 60 %) e da própria chuva.

Referência. Dados ϕ' e h, a falha é $c' < c'_{\mathrm{crit}} = \tau - (\sigma_n - \chi\gamma_w h)\tan\phi'$, e c' condicionada a ϕ' é lognormal. Então

$$P_f = \iint \varphi(z_2)\, f_h(h)\, \Phi\left(\frac{\ln c'_{\mathrm{crit}} - \lambda_c - \zeta_c \rho_Z z_2}{\zeta_c \sqrt{1-\rho_Z^2}}\right) dz_2\, dh,$$

com o integrando nulo onde $c'_{\mathrm{crit}} \le 0$. Essa integral 2D é calculada com quadratura adaptativa até $P_f \sim 10^{-12}$.

6.2 Dois cenários

Tabela 3: Problema 2: os cenários antes (A) e depois (B) da chuva. CoV para Monte Carlo e importância; erro de P_f para o FORM.

	Cenário A: $\mu_h = -2{,}0\,\text{m}$			Cenário B: $\mu_h = -0{,}5\,\text{m}$		
Método	N	P_f	CoV ou erro	N	P_f	CoV ou erro
Referência (integração)	—	$1{,}535 \times 10^{-4}$	—	—	0,1215	—
FORM (HL-RF com Armijo)	149	$1{,}990 \times 10^{-4}$	29,7 %	49	0,1405	15,7 %
Monte Carlo	10^3	0	—	10^3	0,115	8,8 %
Monte Carlo	10^6	$1{,}45 \times 10^{-4}$	8,3 %	10^6	0,121	0,3 %
Importância (ponto de projeto)	10^3	$1{,}512 \times 10^{-4}$	7,0 %	10^3	0,1200	4,3 %
Importância (ponto de projeto)	10^4	$1{,}509 \times 10^{-4}$	2,2 %	10^4	0,1208	1,3 %
β do FORM		3,5414			1,0781	
FS com as médias		1,644			1,219	
α^2 de c', ϕ', h		0,13; 0,18; 0,69			0,23; 0,12; 0,65	
Ponto de projeto x^*		6,60 kPa; 28,0°; −0,53 m			8,22 kPa; 29,7°; −0,07 m	

- **Cenário B** (depois da chuva). $P_f = 0{,}121$, da mesma ordem dos 11,5 % do talude do artigo (o talude infinito não é o mesmo talude, mas os parâmetros são). O FS com as médias é 1,219.
  - Monte Carlo com $N = 1000$ basta: CoV de cerca de 9 %.
  - O FORM superestima P_f em 15,7 %.
- **Cenário A** (antes da chuva). $P_f = 1{,}53 \times 10^{-4}$, e o FS com as médias é 1,644.
  - Monte Carlo com $N = 1000$ dá zero; com 10^6 amostras, o CoV ainda é de 8 %.
  - A amostragem por importância chega a um CoV de 7,0 % com 1000 amostras.
  - O FORM erra 29,7 %: o produto $(\sigma_n - \gamma_w h)\tan\phi'$ curva a superfície no espaço U (Figura 3).
- **Fatores de importância.** Nos dois cenários, a carga de pressão h domina ($\alpha_h^2 \approx 0{,}65$ a 0,69), mais do que c' e ϕ' juntos. No talude, a variável da chuva pesa mais que a resistência.

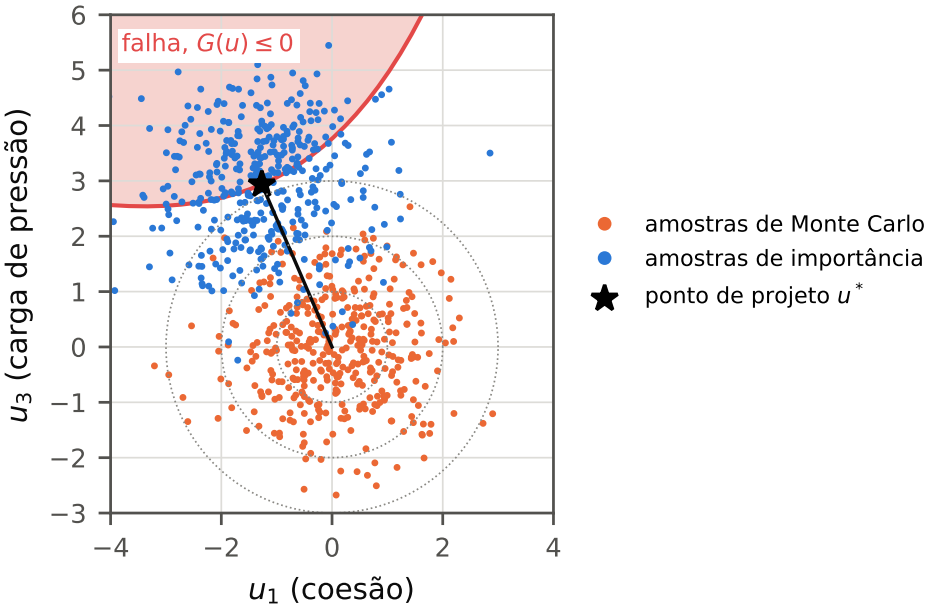


Figura 3: Cenário A no plano (u_1, u_3) (coesão e carga de pressão), com u_2 no valor do ponto de projeto: a superfície de falha e 400 amostras de cada método. Monte Carlo amostra em torno da origem, e quase nenhuma amostra falha. A amostragem por importância amostra em torno de $\boldsymbol{u}^*$, e cerca de metade das amostras falha.

6.3 O efeito dos erros do código original

Com o erro 1, o código usa $\boldsymbol{z} = L^{-1}\boldsymbol{y}$ na volta. A covariância implícita das Z passa a ser $L^{-1}L^{-T}$, e não R_Z. Com $\rho = -0{,}5$, a correlação implícita é $+0{,}459$ (o sinal se inverte), e o desvio de z_2 é 1,314. A Tabela 4 mostra o efeito no FORM:

- com o erro 1, P_f fica 30,2 vezes maior no cenário A e 1,5 vez maior no B;
- sem a correção de Nataf (erro 6), a diferença é pequena, porque os CoV são pequenos;
- a correlação negativa entre c' e ϕ' protege: com $\rho = 0$, o P_f de referência fica 6 vezes maior no cenário A.

Tabela 4: FORM com a transformação certa e com os erros do código original, e o caso independente.

	Cenário A		Cenário B	
Transformação usada no FORM	β	P_f do FORM	β	P_f do FORM
correta (Nataf, $u = L^{-1}z$)	3,5414	$1{,}99 \times 10^{-4}$	1,0781	0,140
com o erro do `FromXtoY` (L no lugar de L^{-1})	2,5112	$6{,}02 \times 10^{-3}$	0,8076	0,210
sem a correção de Nataf ($R_Z = R_X$)	3,5239	$2{,}13 \times 10^{-4}$	1,0733	0,142
c' e ϕ' independentes ($\rho = 0$), FORM	3,0658	$1{,}09 \times 10^{-3}$	0,9468	0,172
c' e ϕ' independentes, referência	3,1067	$9{,}46 \times 10^{-4}$	0,9995	0,159

6.4 HL-RF com e sem a busca de Armijo

O HL-RF puro oscila em torno do ponto de projeto e não converge para $\mu_h \le -2{,}50$ m. Ali, depois de 100 iterações, o β está errado em até 30 % (Tabela 5). Com a busca de Armijo, ele converge em 9 a 21 iterações nesses cenários. Num modelo de elementos finitos, cada iteração do HL-RF puro que não converge custa $2n + 1$ análises.

Tabela 5: O HL-RF puro e com a busca de Armijo ao longo dos cenários de chuva.

μ_h (m)	FS médio	P_f (referência)	β (HL-RF com Armijo)	iterações	β (HL-RF puro)	iterações
−4,00	2,210	$2{,}86 \times 10^{-12}$	6,8361	21	4,8949	não converge (100)
−3,50	2,069	$6{,}25 \times 10^{-10}$	6,0200	15	4,7444	não converge (100)
−3,00	1,927	$7{,}39 \times 10^{-8}$	5,1975	12	4,5865	não converge (100)
−2,50	1,785	$4{,}65 \times 10^{-6}$	4,3705	9	4,3699	não converge (100)
−2,00	1,644	$1{,}53 \times 10^{-4}$	3,5414	21	3,5414	35
−1,00	1,361	0,0243	1,8900	11	1,8900	11
0,00	1,077	0,350	0,2877	4	0,2877	4
0,50	0,936	0,642	−0,4639	5	−0,4639	5

6.5 Curva de fragilidade

A Figura 4 mostra $P_f(\mu_h)$, de 3×10^{-12} ($\mu_h = -4$ m) a 0,64 ($\mu_h = 0{,}5$ m):

- o FORM superestima P_f em 6 % ($\mu_h = 0{,}5$ m) a 42 % ($\mu_h = -4$ m). O erro cresce nos cenários secos, em que β é maior e a curvatura da superfície pesa mais;
- a amostragem por importância com 1000 amostras fica a até 2 desvios da referência em todos os níveis, com CoV de 3 a 10 %. Isso exige um bom ponto de projeto, e o do FORM com Armijo é bom;
- Monte Carlo com 1000 amostras só vê $P_f \gtrsim 10^{-2}$;
- a importância de c' cresce quando a sucção acaba ($\mu_h \to 0$).

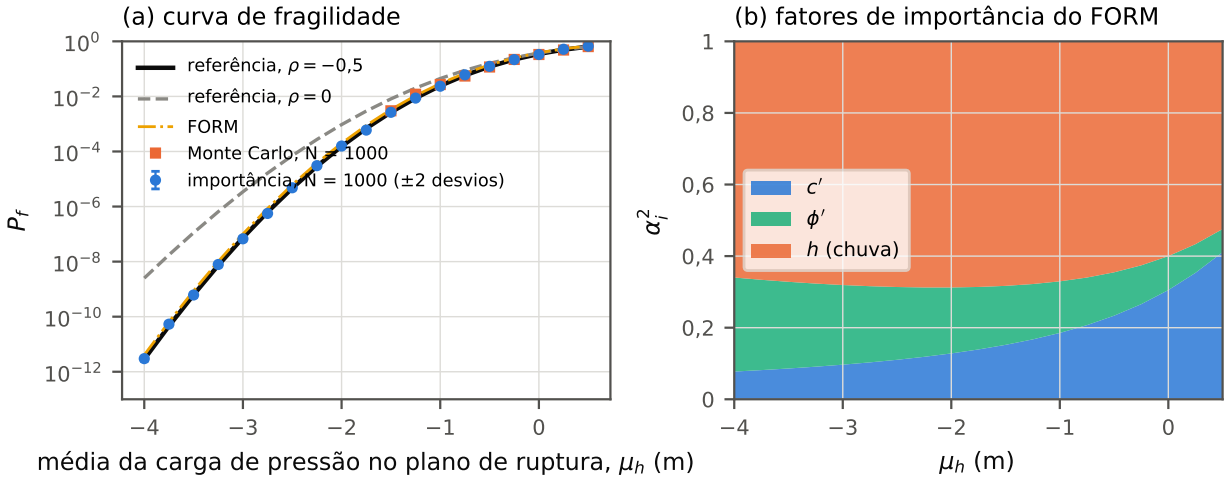


Figura 4: Problema 2. (a) A curva de fragilidade $P_f(\mu_h)$: referência com $\rho = -0{,}5$ e com $\rho = 0$, FORM, importância e Monte Carlo com 1000 amostras. (b) Os fatores de importância do FORM.

7 Uso no talude com chuva e campos aleatórios

7.1 A correspondência

No `SlopeMonteCarlo` (e no `SlopeRandomField`), cada realização tira os coeficientes ξ_k da expansão de Karhunen-Loève dos três campos (c', ϕ' e k_v; `RandomFields::Sample`). Cada campo é $c'(\boldsymbol{x}) = \exp(\lambda + \zeta Z(\boldsymbol{x}))$, com $Z = \sum_k \sqrt{\lambda_k}\xi_k f_k(\boldsymbol{x})$. O fator Γ vem do GIM. A correspondência com as seções anteriores:

Tabela 6: O talude infinito e o talude de elementos finitos.

	Problema 2	Talude com campos aleatórios
Espaço $\boldsymbol{U}$	(u_c, u_ϕ, u_h), $n = 3$	os ξ_k dos três campos: $n = 3M = 6000$ com $M = 2000$. Já são normais padrão independentes: não há transformação de Nataf a fazer
Marginais	lognormal e normal	lognormal ponto a ponto (`klsep::Lognormal`), exata
Correlação c'–ϕ'	ρ_Z e Cholesky	os campos são independentes. Com correlação cruzada, mesmo núcleo: $(\xi_{c,k}, \xi_{\phi,k}) = L_2(\eta_{1,k}, \eta_{2,k})$ em cada modo, com L_2 o Cholesky de $[1\ \rho_Z;\ \rho_Z\ 1]$ (o `ComputeJYZ` corrigido, modo a modo)
$G(\boldsymbol{u})$	$FS - 1$, Eq. (9)	$\Gamma(\boldsymbol{\xi}) - 1$: uma análise (percolação, GIM e refinamento)
Chuva	μ_h	a intensidade e a duração do transiente; $P_f(t)$ ou P_f por cenário
Custo de G	microssegundos	cerca de 95 s por realização no fermi, com 4 níveis (`SlopeMonteCarlo/cluster/README.md`)

7.2 Qual método e quando

- $P_f \gtrsim 5\,\%$ (a configuração de referência do artigo: 11,5 %; o nosso estudo de refinamento: 10,8 %). Monte Carlo basta:
  - $N = 1000$ dá CoV de 9 %, e $N = 10\,000$, cerca de 3 %;
  - a amostragem por importância reduziria N de 2 a 4 vezes (no cenário B, $1{,}8 \times 10^2$ contra $7{,}2 \times 10^2$ realizações para um CoV de 10 %), mas exige achar o deslocamento.
- $P_f \lesssim 10^{-2}$: outros cenários, como menos chuva, um talude menos íngreme ou um solo mais resistente; probabilidades de projeto ($\beta \approx 2{,}5$ a 3); e o começo da curva $P_f(t)$.
  - Monte Carlo para um CoV de 10 % precisa de $N \approx 100/P_f$ realizações: 10^5 análises para $P_f = 10^{-3}$, cerca de 2600 h de processo no fermi.
  - A amostragem por importância precisa de algumas centenas: no cenário A, $4{,}9 \times 10^2$ contra $6{,}5 \times 10^5$.
- **FORM direto no modelo de elementos finitos: não.**
  - O gradiente por diferenças custa $2n$ análises por iteração (centrais) ou $n+1$ (progressivas). Com $n = 6000$ e diferenças progressivas, são cerca de $6000 \times 95\,\text{s} \approx 6{,}6$ dias de processo por iteração.
  - O Γ do GIM tem a tolerância do próprio método. O passo das diferenças teria de ser grande (por exemplo, 0,1 a 0,3 em u), ou o gradiente sai ruído.
  - O FORM só é viável com poucos modos ($M = 20$ a 50 por campo, $n \leq 150$): cerca de 4 h por iteração num processo, ou minutos com as análises em paralelo. Serve para os fatores de importância (quais campos e quais modos pesam) e como ponto de partida da amostragem por importância.

7.3 Amostragem por importância sem gradiente: o caminho proposto

O deslocamento ótimo de uma normal com covariância identidade, no sentido da entropia cruzada (Rubinstein e Kroese, 2004), é a média condicional à falha: $\boldsymbol{u}^\dagger = \mathrm{E}[\boldsymbol{U} \mid G(\boldsymbol{U}) \leq 0]$. Ela pode ser estimada com as realizações que já existem:

1. **Piloto.** Use as realizações de Monte Carlo já feitas. Os $\boldsymbol{\xi}$ de cada uma são reprodutíveis pela semente (`klsep::Normal(semente · 0x9E37... + 3 real + campo)`).
2. **Deslocamento.** Calcule a média dos $\boldsymbol{\xi}$ das realizações com $\Gamma < 1$. Se forem poucas, use as 10 % com menor Γ, como um nível intermediário da entropia cruzada: o GIM dá Γ contínuo, e os níveis $\Gamma \le \gamma_1$, com $\gamma_1 > \gamma_2 > \cdots > 1$, saem de graça.
3. **Regularização.** Mantenha o deslocamento só nos K primeiros modos de cada campo (os de maior λ_k; $K = 20$ a 50) e zere os outros. Cada componente da média tem ruído de cerca de $1/\sqrt{N_f}$. Com $n = 6000$ componentes, $\|\boldsymbol{\delta}\|^2 \approx n/N_f$, e o $\mathrm{E}[w^2]$ cresceria como $e^{\|\boldsymbol{\delta}\|^2}$ (Seção 2.5).
4. **Amostragem.** Rode N_{IS} realizações com $\boldsymbol{\xi} = \boldsymbol{u}^\dagger + \boldsymbol{\varepsilon}$. Grave Γ e $\ln w = -\boldsymbol{\xi}\cdot\boldsymbol{u}^\dagger + \frac{1}{2}\|\boldsymbol{u}^\dagger\|^2$. Daí $\hat{P}_f = \frac{1}{N}\sum I[\Gamma < 1]\, w$, e o CoV vem da amostra.
5. **Iteração.** Se o piloto não tiver falhas, repita os passos 2 a 4 com níveis de Γ cada vez mais baixos (entropia cruzada em vários níveis), ou use a simulação por subconjuntos (Au e Beck, 2001). Ela não precisa de deslocamento: cerca de N análises por nível e $\log_{10}(1/P_f)$ níveis.

A mudança no código. Ela é pequena e fica em `RandomFields::Sample`, no arquivo `Projects/Common/SlopeFields.cpp`:

- depois de sortear `fXi`, some o deslocamento, lido de um arquivo (por exemplo, `DESLOCAMENTO` com 3 colunas e M linhas);
- devolva $\ln w$ e grave-o no CSV da realização, ao lado de Γ;
- a estatística do `SlopeMonteCarlo` passa a usar $\sum Iw/N$ em vez de n_f/N.

Uma vantagem para a chuva. Os pesos não dependem do instante. Com Γ calculado em vários instantes do transiente, uma única amostragem dá $P_f(t)$ em todos eles. Ela é mais eficiente perto do instante para o qual o deslocamento foi escolhido.

O talude infinito como teste. O problema 2 serve para ensaiar a amostragem antes de gastar análises: um modelo barato com os mesmos parâmetros, que dá a ordem de grandeza de P_f por cenário e mostra quais variáveis pesam (lá, a carga de pressão). O ajuste lognormal de Γ do `SlopeMonteCarlo` extrapola P_f de graça, mas depende do modelo de distribuição. A amostragem por importância não tem viés.

8 Conclusões

- O FORM do código original estava certo ($\beta = 3{,}049$), mas a transformação $x \leftrightarrow y$ tinha L e L^{-1} trocados e a volta errada. Com correlação, isso muda P_f por um fator de até 30 no problema proposto.
- O código revisado separa o modelo probabilístico (Nataf), a função de estado limite (escrita uma vez, nas variáveis físicas) e os métodos (FORM com Armijo, Monte Carlo e importância, com o CoV).
- Com $P_f \sim 10^{-3}$, Monte Carlo com 1000 amostras não serve. A amostragem por importância no ponto de projeto dá CoV de 6 % com as mesmas 1000.
- No talude infinito sob chuva, a carga de pressão domina P_f. O FORM superestima P_f em 6 a 42 %, e o HL-RF sem a busca de Armijo não converge nos cenários secos.
- No talude de elementos finitos, com $P_f \approx 11\,\%$, Monte Carlo basta. Para P_f pequenos, o caminho é a amostragem por importância com o deslocamento estimado das realizações que falham, restrito aos primeiros modos, ou a simulação por subconjuntos. O FORM direto não é viável com 6000 variáveis.

Referências

[1] Au, S.-K.; Beck, J. L. (2001). Estimation of small failure probabilities in high dimensions by subset simulation. *Probabilistic Engineering Mechanics* 16(4), 263–277.

[2] Au, S.-K.; Beck, J. L. (2003). Importance sampling in high dimensions. *Structural Safety* 25(2), 139–163.

[3] Hasofer, A. M.; Lind, N. C. (1974). Exact and invariant second-moment code format. *Journal of the Engineering Mechanics Division* (ASCE) 100(1), 111–121.

[4] Liu, P.-L.; Der Kiureghian, A. (1986). Multivariate distribution models with prescribed marginals and covariances. *Probabilistic Engineering Mechanics* 1(2), 105–112.

[5] Lu, N.; Godt, J. (2008). Infinite slope stability under steady unsaturated seepage conditions. *Water Resources Research* 44, W11404.

[6] Melchers, R. E.; Beck, A. T. (2018). *Structural Reliability Analysis and Prediction*, 3. ed. Wiley.

[7] Rackwitz, R.; Fiessler, B. (1978). Structural reliability under combined random load sequences. *Computers & Structures* 9(5), 489–494.

[8] Rubinstein, R. Y.; Kroese, D. P. (2004). *The Cross-Entropy Method*. Springer.

[9] Vargas Ceron, Cecílio, Linn e Maghous (2025). O artigo de referência do `SlopeRandomField` e do `SlopeMonteCarlo` (Seção 6 e Tabela 2).

[10] Zhang, Y.; Der Kiureghian, A. (1995). Two improved algorithms for reliability analysis. Em: Rackwitz, R.; Augusti, G.; Borri, A. (eds.), *Reliability and Optimization of Structural Systems*. Springer.

A Como reproduzir

- **Mathematica:** abra `confiabilidade.wl` e use *Run All Code*. Os valores esperados estão nos comentários. As estimativas de Monte Carlo e de importância mudam com a semente, dentro do CoV impresso.
- **Python:** em `python/`, `python3 problemas.py` (cerca de 2 min) grava `doc/tabelas`, `doc/figuras` e `dados/p2_fragilidade.csv`. Depois, em `doc/`, `latexmk -pdf confiabilidade.tex`.

B O código revisado

```
(* ::Package:: *)

(* ::Title:: *)
(*Confiabilidade: FORM (HL-RF), Monte Carlo e amostragem por importancia*)


(* ::Text:: *)
(*Versao revisada do codigo original: a lista dos erros corrigidos e a teoria estao em doc/
    confiabilidade.pdf.*)
(*Convencoes:*)
(*  X  variaveis fisicas, com marginais F_i e correlacao RX;*)
(*  Z  normais padrao correlacionadas, z_i = Phi^-1(F_i(x_i)), com correlacao RZ (a de Nataf);
    *)
(*  U  normais padrao independentes, z = L.u com RZ = L.Transpose[L] (L triangular inferior); u
     = L^-1.z.*)
(*Falha: g(x) <= 0; no espaco U, G(u) = g(x(u)).*)
(*Os numeros de referencia nos comentarios vem da versao em Python (python/confiab.py), o mesmo
     algoritmo; as*)
```

```
(*estimativas de Monte Carlo e de importancia mudam com a semente, dentro do coeficiente de variacao impresso.*)


ClearAll["Global`*"];
SeedRandom[20261114];
normal01 = NormalDistribution[0, 1];


(* ::Section:: *)
(*1. Marginais e transformacao de Nataf*)


(* lognormal pela media e pelo coeficiente de variacao (o Mathematica usa os parametros de ln X) *)
lnDist[media_, cov_] := LogNormalDistribution[Log[media] - Log[1 + cov^2]/2, Sqrt[Log[1 + cov^2]]];

(* x = F^-1(Phi(z)), a transformacao exata: formas fechadas (vetorizadas) para a normal e a lognormal; para as outras
   marginais, ponto a ponto, com a cauda superior pela funcao de sobrevivencia (Phi(z) perto de 1 perde precisao) *)
xDeZ[NormalDistribution[m_, s_], z_] := m + s z;
xDeZ[LogNormalDistribution[m_, s_], z_] := Exp[m + s z];
xDeZ[d_, z_List] := xDeZ[d, #] & /@ z;
xDeZ[d_, z_?NumericQ] := If[z <= 0, InverseCDF[d, CDF[normal01, z]], InverseSurvivalFunction[d, CDF[normal01, -z]]];

zDeX[NormalDistribution[m_, s_], x_] := (x - m)/s;
zDeX[LogNormalDistribution[m_, s_], x_] := (Log[x] - m)/s;
zDeX[d_, x_?NumericQ] := InverseCDF[normal01, CDF[d, x]];

(* correlacao no espaco Z que reproduz a correlacao r das variaveis fisicas (Liu e Der Kiureghian 1986): formas fechadas
   para normal-normal, lognormal-lognormal e normal-lognormal; para outras marginais, a equacao integral *)
cvDe[d_] := StandardDeviation[d]/Mean[d];
rhoZ[_, _, 0 | 0.] := 0.;
rhoZ[_NormalDistribution, _NormalDistribution, r_] := r;
rhoZ[di_LogNormalDistribution, dj_LogNormalDistribution, r_] := Log[1 + r cvDe[di] cvDe[dj]]/(di[[2]] dj[[2]]);
rhoZ[_NormalDistribution, dj_LogNormalDistribution, r_] := r cvDe[dj]/dj[[2]];
rhoZ[di_LogNormalDistribution, _NormalDistribution, r_] := r cvDe[di]/di[[2]];
rhoZ[di_, dj_, r_] := Module[{rx, s},
   rx[s_?NumericQ] := NIntegrate[
      (xDeZ[di, z1] - Mean[di]) (xDeZ[dj, s z1 + Sqrt[1 - s^2] z2] - Mean[dj]) PDF[normal01, z1] PDF[normal01, z2],
      {z1, -8, 8}, {z2, -8, 8}, Method -> {Automatic, "SymbolicProcessing" -> 0}]/
     (StandardDeviation[di] StandardDeviation[dj]);
   s /. FindRoot[rx[s] == r, {s, r, 0.95 r}]];

(* modelo probabilistico: marginais, correlacoes e o fator de Cholesky *)
modelo[dists_List, rx_?MatrixQ] := Module[{n = Length[dists], rz, L},
   rz = Table[If[i == j, 1., rhoZ[dists[[i]], dists[[j]], rx[[i, j]]]], {i, n}, {j, n}];
   L = Transpose[CholeskyDecomposition[rz]];         (* CholeskyDecomposition da U com Transpose[U].U = rz *)
   If[Max[Abs[L . Transpose[L] - rz]] > 10^-10, Print["modelo: a decomposicao de Cholesky falhou"]];
   <|"dists" -> dists, "RX" -> rx, "RZ" -> rz, "L" -> L, "n" -> n|>];
modelo[dists_List] := modelo[dists, IdentityMatrix[Length[dists]]];

(* u -> x de um ponto; us -> xs de muitas amostras (uma por linha, vetorizado); x -> u *)
xDeU[mod_, u_?VectorQ] := MapThread[xDeZ, {mod["dists"], mod["L"] . u}];
xDeUs[mod_, us_?MatrixQ] := Transpose[MapThread[xDeZ, {mod["dists"], Transpose[us . Transpose[mod["L"]]]}]];
uDeX[mod_, x_?VectorQ] := LinearSolve[mod["L"], MapThread[zDeX, {mod["dists"], x}]];


(* ::Section:: *)
(*2. FORM: HL-RF com busca de Armijo (Zhang e Der Kiureghian 1995)*)


(* gradiente por diferencas centrais: 2n avaliacoes de G (num modelo de elementos finitos, cada uma e uma analise) *)
```

```
gradiente[G_, u_, h_] := Table[(G[u + h UnitVector[Length[u], i]] - G[u - h UnitVector[Length[u], i]])/(2 h),
   {i, Length[u]}];

(* Passo: o h das diferencas; TolG: |G|/|G(0)|; TolU: |u - (alfa.u) alfa|, u alinhado com o gradiente;
   Armijo -> False: o HL-RF puro (pode oscilar sem convergir; ver o problema 2 com mu_h <= -2,5 m) *)
Options[form] = {"Passo" -> 10.^-5, "TolG" -> 10.^-6, "TolU" -> 10.^-4, "MaxIter" -> 100, "Armijo" -> True};
form[G_, n_Integer, OptionsPattern[]] := Module[
   {h = OptionValue["Passo"], u = ConstantArray[0., n], g, g0, dg, ndg, alfa = ConstantArray[0., n], d, c, m0, dm,
    lam, un, gn, it = 0, nav = 0, conv = False, hist = {}, beta},
   g = G[u]; nav++;
   g0 = If[g == 0, 1., Abs[g]];
   While[it < OptionValue["MaxIter"],
    it++;
    dg = gradiente[G, u, h]; nav += 2 n;
    ndg = Norm[dg]; alfa = -dg/ndg;
    AppendTo[hist, {it, u, g, Norm[u]}];
    If[Abs[g]/g0 < OptionValue["TolG"] && Norm[u - (alfa . u) alfa] < OptionValue["TolU"], conv = True; Break[]];
    d = ((dg . u - g)/ndg^2) dg - u;                    (* do ponto atual ao ponto HL-RF *)
    lam = 1.;
    If[TrueQ[OptionValue["Armijo"]],
     (* merito m(u) = |u|^2/2 + c|G(u)|, com c > |u|/|grad G| para d ser direcao de descida *)
     c = (2 Norm[u] + 1.)/ndg;
     m0 = u . u/2 + c Abs[g];
     dm = (u + c Sign[g] dg) . d;
     While[True,
      un = u + lam d; gn = G[un]; nav++;
      If[un . un/2 + c Abs[gn] <= m0 + 10.^-4 lam dm || lam < 2.^-10, Break[]];
      lam = lam/2],
     un = u + d; gn = G[un]; nav++];
    u = un; g = gn];
   beta = alfa . u;                                     (* com sinal: negativo se a origem esta na falha *)
   <|"beta" -> beta, "u" -> u, "alfa" -> alfa, "pf" -> CDF[normal01, -beta], "iteracoes" -> it, "avaliacoes" -> nav,
    "convergiu" -> conv, "historico" -> hist|>];


(* ::Section:: *)
(*3. Monte Carlo e amostragem por importancia*)


(* Gv: G vetorizada, recebe as amostras (uma por linha) e devolve a lista dos valores de G *)
monteCarlo[Gv_, n_Integer, nAm_Integer] := Module[{q, pf},
   q = N[Boole[Thread[Gv[RandomVariate[normal01, {nAm, n}]] <= 0]]];
   pf = Mean[q];
   <|"pf" -> pf, "cov" -> If[pf > 0, Sqrt[(1 - pf)/(nAm pf)], Infinity],
    "beta" -> If[0 < pf < 1, -InverseCDF[normal01, pf], Indeterminate], "q" -> q|>];

(* densidade de amostragem h = N(ustar, I): q = I(G <= 0) phi(v)/phi(v - ustar), com ln w = -v.ustar + |ustar|^2/2 *)
importancia[Gv_, ustar_?VectorQ, nAm_Integer] := Module[{v, q, pf},
   v = RandomVariate[normal01, {nAm, Length[ustar]}] + ConstantArray[ustar, nAm];
   q = N[Boole[Thread[Gv[v] <= 0]]] Exp[-v . ustar + ustar . ustar/2];
   pf = Mean[q];
   <|"pf" -> pf, "cov" -> If[pf > 0, StandardDeviation[q]/(Sqrt[nAm] pf), Infinity],
    "beta" -> If[0 < pf < 1, -InverseCDF[normal01, pf], Indeterminate], "q" -> q|>];

mediaAcumulada[q_List] := Accumulate[q]/Range[Length[q]];
(* pontos espacados em escala log para os graficos de convergencia *)
amostraLog[lista_List] := With[{k = Union[Round[10^Range[0., Log10[Length[lista]], 0.02]]]}, Transpose[{k, lista[[k]]}]];


(* ::Section:: *)
(*4. Problema 1: g = X1 X2 - X3 (o do codigo original)*)


mod1 = modelo[{NormalDistribution[40, 5], NormalDistribution[50, 2.5], NormalDistribution[1000, 200]}];
```

```
g1X[{x1_, x2_, x3_}] := x1 x2 - x3;            (* vale para um ponto ou para as colunas de muitas amostras *)
g1U[u_?VectorQ] := g1X[xDeU[mod1, u]];
g1Us[us_?MatrixQ] := g1X[Transpose[xDeUs[mod1, us]]];

(* referencia: P(X3 >= X1 X2), integrando nas duas primeiras variaveis *)
pfRef1 = NIntegrate[PDF[normal01, u1] PDF[normal01, u2] CDF[normal01, (1000 - (40 + 5 u1) (50 + 2.5 u2))/200],
   {u1, -9, 9}, {u2, -9, 9}, PrecisionGoal -> 8];
Print["referencia: pf = ", pfRef1, ", beta = ", -InverseCDF[normal01, pfRef1]];   (* 1.17688*10^-3 e 3.0415 *)

f1 = form[g1U, 3];
Print["FORM: beta = ", f1["beta"], ", pf = ", f1["pf"], ", u* = ", f1["u"], ", x* = ", xDeU[mod1, f1["u"]],
  ", alfa^2 = ", f1["alfa"]^2, "; ", f1["iteracoes"], " iteracoes, ", f1["avaliacoes"], " avaliacoes de g"];
(* beta = 3.04907, pf = 1.1477*10^-3 (2,5 % abaixo da referencia), u* = {-2.28993, -0.676678, 1.8961},
   x* = {28.55, 48.31, 1379.2}, alfa^2 = {0.564, 0.049, 0.387}; 6 iteracoes, 43 avaliacoes *)

mc1 = monteCarlo[g1Us, 3, 10^6];
is1 = importancia[g1Us, f1["u"], 10^3];
Print["Monte Carlo, N = 10^6: pf = ", mc1["pf"], " (CoV ", mc1["cov"], ")"];          (* CoV ~ 0,03 *)
Print["importancia, N = 10^3: pf = ", is1["pf"], " (CoV ", is1["cov"], ")"];          (* CoV ~ 0,06 *)
(* Monte Carlo com N = 1000 (o do codigo original): CoV ~ 0,92 e, em 31 % das rodadas, nenhuma falha (pf = 0) *)

qmc = monteCarlo[g1Us, 3, 10^5]["q"];
qis = importancia[g1Us, f1["u"], 10^5]["q"];
ListLogLinearPlot[{amostraLog[mediaAcumulada[qmc]], amostraLog[mediaAcumulada[qis]]}, Joined -> True,
 PlotRange -> {0, 0.003}, GridLines -> {None, {pfRef1, f1["pf"]}},
 PlotLegends -> {"Monte Carlo", "importancia (ponto de projeto)"}, AxesLabel -> {"N", "Pf acumulada"}]


(* ::Section:: *)
(*5. Problema 2 (proposto): talude infinito sob chuva*)


(* Plano de ruptura paralelo a superficie, a profundidade zf; Bishop com chi = 1 (phi^b = phi'), como no
   SlopeRandomField. h = carga de pressao no plano de ruptura (m): h < 0 succao, h > 0 pressao positiva.
   Parametros do talude de Vargas Ceron et al. (2025): beta = 45 graus, c' LN(10 kPa; 30 %), phi' LN(30 graus; 10 %).
   A chuva entra pela media mu_h de h: mais chuva, maior mu_h. *)
gama = 20.; gamaW = 9.81; betaS = 45. Degree; zf = 2.; sigH = 0.5;
sigN = gama zf Cos[betaS]^2; tau = gama zf Sin[betaS] Cos[betaS];    (* 20 kPa e 20 kPa *)
chi[h_] := 1;
(* alternativa: chi = Se(h) de van Genuchten, por exemplo com alfaVG = 1/m e nVG = 1.6:
   chi[h_] := UnitStep[h] + (1 - UnitStep[h]) (1 + (alfaVG Abs[h])^nVG)^(-(1 - 1/nVG)) *)
fsTalude[{c_, phi_, h_}] := (c + (sigN - gamaW h chi[h]) Tan[phi Degree])/tau;
g2X[x_] := fsTalude[x] - 1;

mod2[muh_, rho_: -0.5] := modelo[{lnDist[10., 0.3], lnDist[30., 0.1], NormalDistribution[muh, sigH]},
   {{1, rho, 0}, {rho, 1, 0}, {0, 0, 1}}];

(* referencia: dado phi' e h, c' e lognormal (condicionada a phi'), e a falha e c' < c'_crit(phi', h) *)
pfRef2[muh_, rho_: -0.5] := Module[{dc = lnDist[10., 0.3], dp = lnDist[30., 0.1], rz, cc},
   rz = rhoZ[dc, dp, rho];
   cc[z2_, h_] := tau - (sigN - gamaW h chi[h]) Tan[xDeZ[dp, z2] Degree];
   NIntegrate[PDF[normal01, z2] PDF[NormalDistribution[muh, sigH], h] *
     If[cc[z2, h] > 0, CDF[normal01, (Log[cc[z2, h]] - dc[[1]] - dc[[2]] rz z2)/(dc[[2]] Sqrt[1 - rz^2])], 0],
    {z2, -8, 8}, {h, muh - 8 sigH, muh + 8 sigH}, PrecisionGoal -> 8,
    Method -> {Automatic, "SymbolicProcessing" -> 0}]];

(* um cenario: referencia, FORM, Monte Carlo e importancia *)
cenario[muh_, rho_: -0.5] := Module[{m = mod2[muh, rho], G, Gv, f},
```

```
197    G[u_?VectorQ] := g2X[xDeU[m, u]];
198    Gv[us_?MatrixQ] := g2X[Transpose[xDeUs[m, us]]];
199    f = form[G, 3];
200    <|"muh" -> muh, "ref" -> pfRef2[muh, rho], "form" -> f, "x*" -> xDeU[m, f["u"]],
201     "mc" -> monteCarlo[Gv, 3, 10^5], "is" -> importancia[Gv, f["u"], 10^3]|>];
202
203 (* cenario A (antes da chuva, mu_h = -2 m): ref 1.535*10^-4; FORM beta 3.5414, pf 1.99*10^-4;
       alfa^2 = {0.13, 0.18, 0.69}
204    cenario B (depois da chuva, mu_h = -0.5 m): ref 0.1215 (o P_f do Ceron e 11,5 %); FORM beta
       1.0781, pf 0.1405 *)
205 cA = cenario[-2.];
206 cB = cenario[-0.5];
207 Table[Print[c["muh"], ":␣ref␣", c["ref"], ";␣FORM␣beta␣", c["form"]["beta"], "␣pf␣", c["form"][
       "pf"], "␣alfa^2␣",
208    c["form"]["alfa"]^2, "␣x*␣", c["x*"], ";␣MC␣", c["mc"]["pf"], "␣(CoV␣", c["mc"]["cov"], ");␣
       IS␣", c["is"]["pf"],
209    "␣(CoV␣", c["is"]["cov"], ")"], {c, {cA, cB}}];
210
211 (* curva de fragilidade P_f(mu_h): referencia (rho = -0,5 e 0), FORM e importancia com 1000
       amostras *)
212 frag = Table[Module[{m = mod2[mu], G, Gv, f, is},
213     G[u_?VectorQ] := g2X[xDeU[m, u]];
214     Gv[us_?MatrixQ] := g2X[Transpose[xDeUs[m, us]]];
215     f = form[G, 3]; is = importancia[Gv, f["u"], 1000];
216     {mu, pfRef2[mu], pfRef2[mu, 0], f["pf"], is["pf"], is["cov"], f["alfa"]^2}], {mu, Range
       [-4., 0.5, 0.25]}];
217 ListLogPlot[{frag[[All, {1, 2}]], frag[[All, {1, 3}]], frag[[All, {1, 4}]], frag[[All, {1,
       5}]]},
218  Joined -> {True, True, True, False}, PlotMarkers -> {None, None, None, Automatic},
219  PlotLegends -> {"referencia,␣rho␣=␣-0,5", "referencia,␣rho␣=␣0", "FORM", "importancia,␣N␣=␣
       1000"},
220  AxesLabel -> {"mu_h␣(m)", "Pf"}, PlotRange -> All]
221 ListLinePlot[Transpose[Table[{r[[1]], #} & /@ r[[7]], {r, frag}]], PlotLegends -> {"c'", "phi'"
       , "h"},
222  AxesLabel -> {"mu_h␣(m)", "alfa^2"}]
223
224
225 (* ::Section:: *)
226 (*6. As funcoes do codigo original, corrigidas*)
227
228
229 (* ::Text:: *)
230 (*Os mesmos nomes e argumentos; as mudancas estao marcadas. data = {{caso, media, dp}, ...},
       caso 1 normal, 2 lognormal;*)
231 (*a normal equivalente de Rackwitz-Fiessler da lognormal depende do ponto x (e deve ser refeita
        a cada iteracao).*)
232
233
234 ComputeXZ[data_, x_] := Module[{caso, mu, sig, p1, p2},
235    {caso, mu, sig} = data;
236    Switch[caso,
237     1, {sig, mu},
238     2, p2 = Sqrt[Log[1 + (sig/mu)^2]]; p1 = Log[mu] - p2^2/2; {x p2, x (1 - Log[x] + p1)}] // N
       ];
239 ComputeJXZ[data_, x_] := Module[{sz = Length[data], sn},
240    sn = Table[ComputeXZ[data[[i]], x[[i]]], {i, sz}];              (* {dp, media} equivalentes
        *)
241    {DiagonalMatrix[sn[[All, 1]]], DiagonalMatrix[1/sn[[All, 1]]], sn[[All, 2]]}];   (* {Jxz,
       Jzx, Mneq} *)
242 (* Y independentes, Z correlacionadas: z = L.y, y = L^-1.z, RX = L.Transpose[L] *)
243 ComputeJYZ[RX_] := Module[{L},
244    L = Transpose[CholeskyDecomposition[N[RX]]];                    (* vale tambem para uma
       variavel: {{1.}} *)
245    If[Max[Abs[L . Transpose[L] - RX]] > 10^-8, Print["Atencao:␣a␣decomposicao␣de␣Cholesky␣
       falhou!"]];   (* era Det *)
246    {Inverse[L], L}];                                                (* {Jyz, Jzy} *)
247 FromXtoY[data_, x_, RX_] := Module[{Jyz, Jzy, Jxz, Jzx, Mneq, Jyx, Jxy},
248    {Jyz, Jzy} = ComputeJYZ[RX];
249    {Jxz, Jzx, Mneq} = ComputeJXZ[data, x];
250    Jyx = Jyz . Jzx;                                                 (* era Jzy.Jzx *)
251    Jxy = Jxz . Jzy;                                                 (* era Jxz.Jyz *)
252    {Jyx . (x - Mneq), Jxy, Jyx, Jxz, Mneq}];
253 FromYtoZ[y_, RX_] := Module[{Jyz, Jzy}, {Jyz, Jzy} = ComputeJYZ[RX]; Jzy . y];   (* nomes na
       ordem certa *)
254 FromZtoX[z_, Jxz_, Mneq_] := Jxz . z + Mneq;                        (* era Jxz.(z + Mneq) *)
```

```
255
256 (* teste de ida e volta com correlacao (com RX = I, o caso do codigo original, o erro nao
        aparece) *)
257 datarv = {{1, 40, 5}, {1, 50, 2.5}, {1, 1000, 200}};
258 RXc = {{1, 0.5, 0}, {0.5, 1, 0}, {0, 0, 1}};
259 x0 = {35., 52., 1100.};
260 {y0, Jxy0, Jyx0, Jxz0, Mneq0} = FromXtoY[datarv, x0, RXc];
261 Print["ida␣e␣volta␣x␣->␣y␣->␣z␣->␣x:␣", FromZtoX[FromYtoZ[y0, RXc], Jxz0, Mneq0], "␣(deve␣dar␣"
        , x0, ")"];
262
263
264 (* ::Section:: *)
265 (*7. Esboco: G de um modelo externo (o talude de elementos finitos)*)
266
267
268 (* ::Text:: *)
269 (*No talude com campos aleatorios, U sao os coeficientes xi da expansao de Karhunen-Loeve (ja
        normais padrao e*)
270 (*independentes) e G(u) = Gamma(u) - 1, com Gamma o fator do GIM. Um G externo teria a forma
        abaixo (nao executado:*)
271 (*o programa e o arquivo sao ilustrativos). Cada avaliacao e uma analise; o gradiente custa 2n
        analises.*)
272
273
274 (*
275 gTalude[u_?VectorQ] := Module[{arq = "xi.csv", saida},
276    Export[arq, u];                                                    (* os xi da realizacao *)
277    saida = RunProcess[{"./SlopeMonteCarlo", "xi", arq}, "StandardOutput"];
278    ToExpression[Last[StringSplit[saida]]] - 1];                       (* Gamma - 1 *)
279 *)
```